Design and Implementation of Checkspeare engine and GUI

A Technical Study of Search Algorithms, Bitboard Optimizations, Opening Books, Cross-Platform Process Communication, and GUI Integration

Technical Project Report

Author: The Check-Mates
Date: October 17, 2026

Abstract

This report documents Checkspeare engine and GUI from their C++23 source. The engine provides UCI communication, bitboard move generation, compile-time-selected PEXT or Hyperbola Quintessence sliding attacks, iterative-deepening alpha-beta search, quiescence, threshold static exchange evaluation, Zobrist hashing, a transposition table, and handcrafted evaluation. The SFML/ImGui GUI links EngineCore for local game state and move validation, and launches the engine as a subprocess for search. The report covers Polyglot book mapping, asynchronous UCI I/O, platform-specific process adapters, tests, and measured performance.

Contents

1 Introduction and Scope **1**

2 System Architecture and Component Responsibilities **1**

3 Board Representation, Attack Generation, and Legal Moves **2**

3.1 Position representation 2

3.2 Attacks and sliding pieces 3

4 Search Algorithms **4**

4.1 Negamax, alpha-beta, and iterative deepening 4

4.2 Quiescence search 7

4.3 Move ordering 8

4.4 Zobrist hashing and transposition table 8

4.5 Time control and search termination 9

5 Position Evaluation **9**

5.1 Handcrafted Evaluation 9

6 Polyglot Opening Book **11**

6.1 Purpose and integration 11

6.2 Record layout 11

6.3 Keys, move validation, and selection 12

6.4 In-memory access and platform boundary 12

7 UCI Protocol and Engine Communication **12**

7.1 Concurrent command handling and search 12

8 GUI Architecture: SFML, Dear ImGui, and ImGui-SFML **13**

8.1 Software Architecture 13

8.2 Responsibilities and layout 15

8.3 Board rendering and user interaction 15

8.4 Frame loop and responsiveness 16

9 Cross-Platform Engine Subprocess Management 16

9.1 Shared process interface 16

9.2 POSIX process creation 17

9.3 Win32 process creation 17

9.4 Background reads and thread safety 17

10 C++ Portability, Conditional Compilation, and CMake 17

10.1 Build targets and platform adapters 17

10.2 Compile-time sliding-attack backend and CPU baseline 18

10.3 CMake configuration 18

11 Testing and Validation 19

12 Performance Evaluation and Experimental Results 19

12.1 Test environment 19

12.2 Match against Stockfish 19 at 2900 Elo 19

12.3 Depth-six perft 21

12.4 Depth-18 search 21

12.5 Summary 21

13 Limitations, Future Improvements, and Conclusion 21

1. Introduction and Scope

A chess engine represents positions, generates legal moves, searches continuations, evaluates positions, and reports a move. A chess application adds the board, user input, visible game state, and engine coordination. Separating these roles keeps the engine usable without a GUI.

The project builds Checkspeare-engine, a UCI-speaking search process, and Checkspeare-GUI, an SFML/ImGui application that launches it through redirected standard input and output. Both targets link EngineCore: the GUI also uses its chess position and move-generation code for local state and legality. UCI defines communication, not board representation or search strategy [20]. CMake names the executable targets `CheckspeareEngine` and `CheckspeareGUI`.

The engine includes bitboards, special-move handling, iterative-deepening search, threshold SEE, hand-crafted evaluation, a weighted-or-best Polyglot book, and clock-based search limits. The GUI adds board interaction, clocks, move history, analysis panels, and sound. Later chapters describe the implementation and its measured performance.

2. System Architecture and Component Responsibilities

Figure 2.1 shows the runtime process boundary: the GUI sends UCI commands and turns engine output into display state. Both executables link EngineCore; the GUI uses it for local positions and legal moves, while search runs in the child engine process.

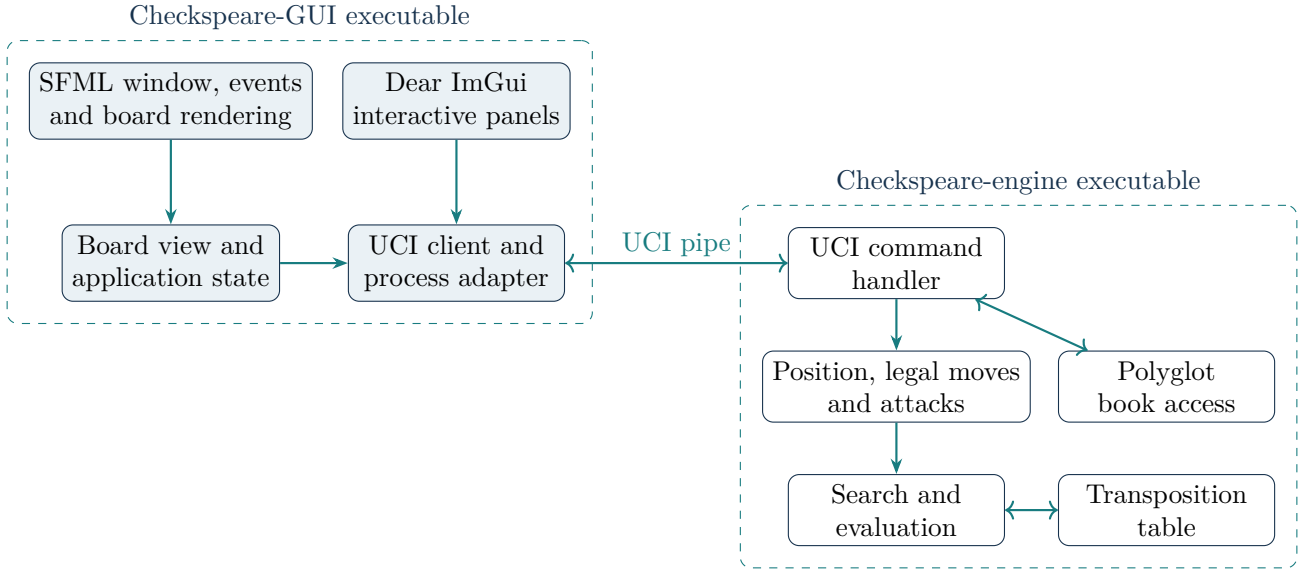


Figure 2.1: Two-process runtime architecture. The GUI links EngineCore for local chess state and exchanges UCI text with the separate search process.

Table 2.1 summarizes component responsibilities.

Table 2.1: Major components and responsibility boundaries.

Component	Main responsibility
Position and move generator	Store chess state; produce and validate candidates, including special moves.
Search and evaluation	Choose a move under limits; reuse prior work and score nonterminal positions.
UCI handler	Parse commands, schedule searches, report analysis and the final move.
Book access	Read Polyglot records and offer validated opening candidates.
UCI client / process adapter	Launch engine, send commands, collect output asynchronously, detect exit.
SFML and board view	Manage window/event/render path and map coordinates to squares.
ImGui panels	Present controls, analysis, configuration, and status.

The process boundary decouples search from rendering and allows another UCI engine. Shared EngineCore keeps local legality consistent; pipes still require serialization, process management, and asynchronous error handling.

3. Board Representation, Attack Generation, and Legal Moves

3.1 Position representation

Square $s \in [0, 63]$ maps to bit s (here, $a1 = 0$); shifts, masks, and attacks must use this convention. Twelve piece bitboards and two color occupancies encode placement, alongside a square-to-piece lookup array. State also stores side to move, castling rights, en-passant square, halfmove clock, and repetition history, but not the FEN fullmove number. Keep the redundant representations synchronized [2].

Bit $s = 8r + f$ represents file f, rank r (so $e2$ is bit 12). Figure 3.1 shows the initial position and white-pawn set; set bits mark membership, not piece identity.

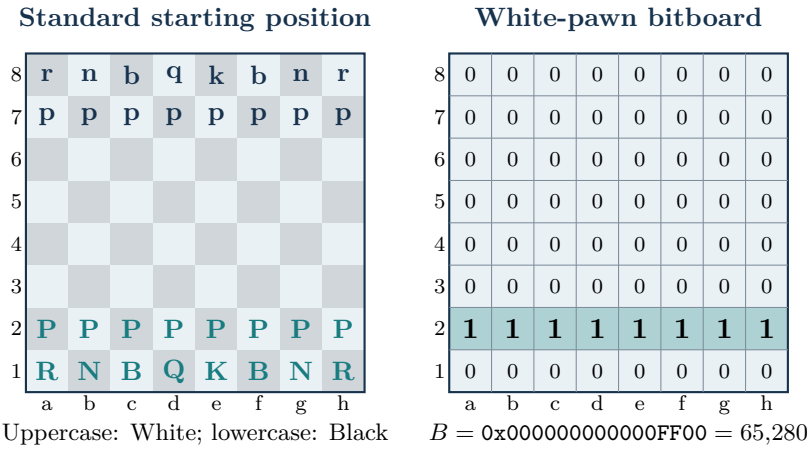


Figure 3.1: Initial chess position and matching white-pawn bitboard; bits 8–15 are set.

The initial white-pawn mask is $B = 2^8 + \cdots + 2^{15} = 0x000000000000FF00 = 65{,}280$. Test bit s with $B \,\&\, (1 \ll s)$ and clear the least-significant set bit with $B \,\&\, (B - 1)$. OR combines piece sets, AND filters targets, and shifts generate candidates; $(P \ll 8) \,\&\, E$ gives empty one-step pawn destinations. Bitboards process squares together; mailbox arrays simplify square lookup.

3.2 Attacks and sliding pieces

Attacks differ from legal moves: pinned pieces still attack, while king destinations require attack checks. Pawn attacks differ from pushes. Knight and king attack tables depend only on the source square; sliding attacks also depend on occupancy and include the first blocker.

3.2.1 PEXT-indexed sliding attacks

For each slider square, a relevant mask M_s selects ray occupancies that can change the attack set; terminal edge squares may be omitted. When the PEXT backend is enabled, BMI2 packs occupied masked bits into a dense index [14]. Figure 3.2 shows the packing and lookup.

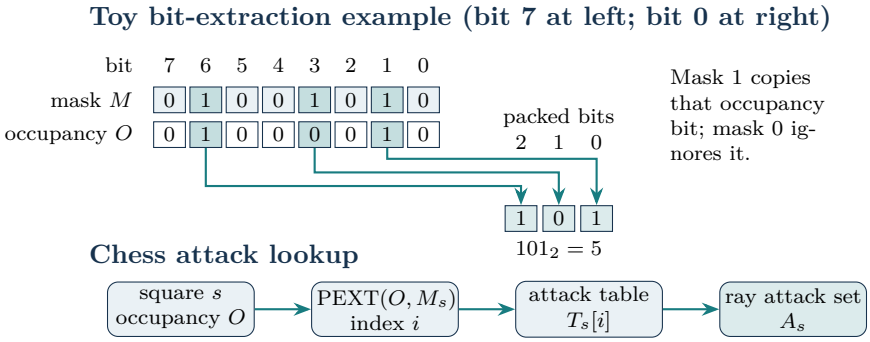


Figure 3.2: PEXT packs occupancy bits 6, 3, and 1 as $101_2 = 5$; that index selects the corresponding attack set from the table.

A mask with k bits requires 2^k precomputed attack entries. At runtime, PEXT(O, M_s) selects an entry; queens combine rook and bishop attacks. This trades memory and setup for fast queries. PEXT requires x86 BMI2, and its speed depends on CPU and cache behavior; benchmark it against HQ on the target machine [14].

3.2.2 Hyperbola Quintessence

Hyperbola Quintessence (HQ) computes sliding attacks with bitboard arithmetic. Mask occupancy O to a rank, file, or diagonal L, and let $R = 2^s$ mark the slider square. A representative expression is

$$A_{line} = [(O - 2R) \oplus \mathrm{rev}(\mathrm{rev}(O) - 2\,\mathrm{rev}(R))] \;\&\; L,$$

Here, `rev` reverses the line's bit order. Unsigned subtraction propagates toward blockers in both directions; XOR combines the rays, and $\&L$ masks the result. Exclude the source square. Indexing conventions matter, so this is a representative formula, not drop-in code [4].

HQ uses arithmetic instead of large occupancy tables; PEXT uses BMI2 tables for fast indexing. Both must return identical attacks. Figure 3.3 compares the paths.

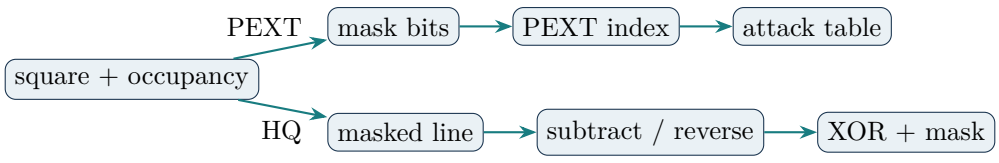


Figure 3.3: Alternative sliding-attack data paths from the same position occupancy.

3.2.3 Backend selection and legal moves

PEXT and HQ are selected at build time; their default, target guards, and ISA requirements are summarized in Section 10.2. The attack API is `bishop_attacks(occupancy, square)`.

The generator creates pseudo-legal moves, detects checkers and pins, then filters candidates. It checks king and en-passant moves by making them and testing king safety; check and pin masks filter many others:

Listing 3.1: Legal-candidate filtering pseudocode.

```
checkers, pinned = find_checkers_and_pinned(position)
for move in pseudo_legal_moves(position):
    if moves_king(move) or move.is_en_passant:
        legal = make_test_unmake_and_check_king(move)
    else:
        legal = satisfies_check_and_pin_masks(move, checkers, pinned)
    if legal: emit(move)
```

Castling requires valid rights, empty transit squares, and unattacked start, transit, and destination squares. En passant must test resulting occupancy because it can uncover a sliding attack; generate all four promotions. Make/unmake restores pieces, rights, the halfmove clock, hash, and cached evaluation. No legal moves means checkmate or stalemate, depending on check status [5].

The move encodes its source, destination, and special-move flags. The matching undo state saves the captured piece, prior castling and en-passant state, halfmove clock, Zobrist key, and cached evaluation totals. Unmake uses the move plus this saved state to restore the position. Assert that occupancies match piece sets, colors do not overlap, and each king appears once outside special test positions.

4. Search Algorithms

4.1 Negamax, alpha-beta, and iterative deepening

Negamax expresses zero-sum minimax by negating the child's score after each move: $N(p, d, \alpha, \beta) = \max_m[-N(p_m, d-1, -\beta, -\alpha)]$.

Alpha-beta tracks bounds α and β, pruning siblings that cannot improve an ancestor's choice without changing the fixed-depth minimax result. Better move ordering increases cutoffs [1]; Figure 4.1 illustrates one.

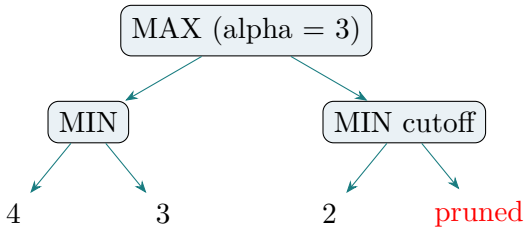


Figure 4.1: Alpha-beta cutoff after the root already has a value of 3.

Iterative deepening completes progressively deeper searches, retaining the last completed move and principal variation (PV). The previous score centers the next iteration's aspiration window; the PV and transposition-table move help order its candidates. Figure 4.2 shows the retry path when the score misses the window. If interrupted, the incomplete iteration is discarded.

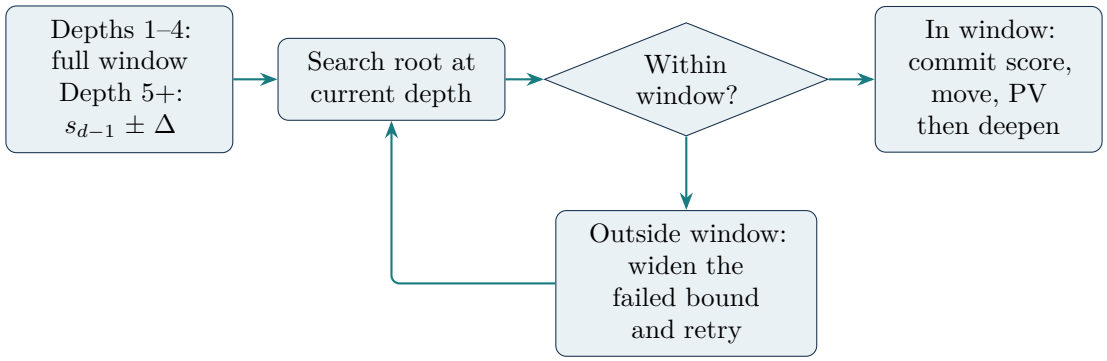


Figure 4.2: Each completed iteration deepens the search. A fail-low widens α downward; a fail-high widens β upward before retrying.

In this implementation, depths one through four use a full window. From depth five, Δ starts at 25 score units and grows by half after each fail-low or fail-high; a sufficiently wide window falls back to the full range. The sketch shows the root control loop:

Listing 4.1: Iterative deepening and aspiration-window pseudocode.

```
best_move = first_legal_move(position)  # safe fallback
previous_score = 0
for depth = 1..depth_limit:
    if depth <= 4:
        score = root_search(depth, -INF, INF)
    else:
        delta = 25
        alpha, beta = previous_score-delta, previous_score+delta
        repeat:
            score = root_search(depth, alpha, beta)
            if score <= alpha: alpha -= delta
            elif score >= beta: beta += delta
            else: break
            delta += delta / 2
            if delta >= 1000: alpha, beta = -INF, INF
        until stopped
    if stopped: break
    best_move, pv = completed_root_result()
    previous_score = score
```

4.1.1 Principal-variation search and selective pruning

At a PV node, the first ordered move receives a full-window search; later moves are tested with a null window, as shown in Figure 4.3. Eligible late quiet moves may first be searched at reduced depth, with the reduction depending on depth, move count, history, and killer/countermove status. A reduced result above α is verified at full depth; captures, promotions, and checked nodes are not reduced.

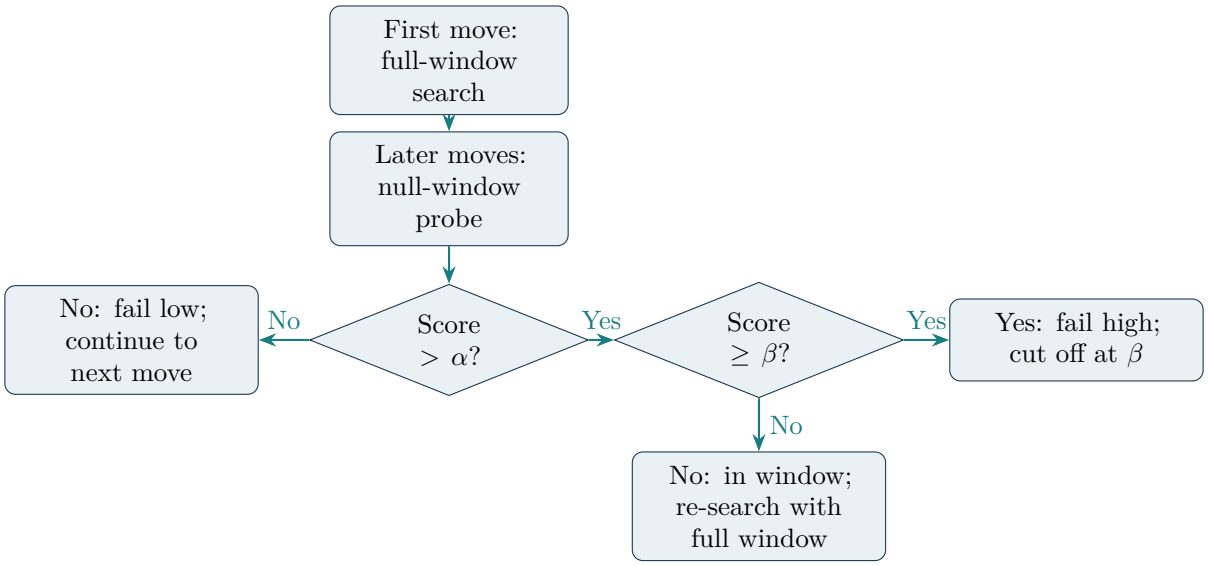


Figure 4.3: Later moves are probed with a null window. Fail-low continues; an in-window score is re-searched; fail-high cuts off.

Listing 4.2: PV move search with null-window probes and late-move reductions.

```
for each ordered legal move:
    make(move)
    if late_quiet(move) and reduction_allowed:
        score = -search(child, depth-1-R, -alpha-1, -alpha)
        if score > alpha:
            score = -search(child, depth-1, -alpha-1, -alpha)
    elif first_move:
        score = -search(child, depth-1, -beta, -alpha)
    else:
        score = -search(child, depth-1, -alpha-1, -alpha)
    if not first_move and alpha < score < beta:
        score = -search(child, depth-1, -beta, -alpha)
    unmake(move)
    best = max(best, score); alpha = max(alpha, score)
    if alpha >= beta: break
```

Other selective rules are guarded by node type, check status, depth, and score margins:

Table 4.1: Selected selective-search mechanisms.

Mechanism	Applicability	Search effect
Check extension	Side to move is in check.	Adds one ply so evasions are resolved before the horizon.
Null-move pruning	Non-PV, not in check, depth at least 3, with non-pawn material and static score at least β.	Searches a reduced null move with a zero-width window; a fail-high can prune the node.
Reverse futility	Non-PV zero-window nodes outside check, depth at most 6.	Returns the static score when it clears β by a depth-scaled margin.
Futility pruning	Shallow non-PV zero-window nodes outside check.	Skips quiet moves unlikely to raise α, except killer and countermoves.
Late-move pruning	Shallow non-PV zero-window nodes after a depth-dependent quiet-move threshold.	Skips additional quiet moves to limit the branching factor.
Late-move reduction	Later quiet moves at sufficient depth, outside check.	Searches less deeply first and verifies at full depth when the reduced result beats α.

These heuristics save work but make the search selective: unlike alpha-beta alone, they can omit parts of the fixed-depth tree. They are guarded approximations, not proofs that a move is inferior.

4.2 Quiescence search

Quiescence mitigates the horizon effect by extending tactically unstable lines beyond the nominal depth; Figure 4.4 shows its quiet-position and in-check paths. Captures are filtered by a victim-value bound. This extension increases node count and can still miss quiet threats [8].

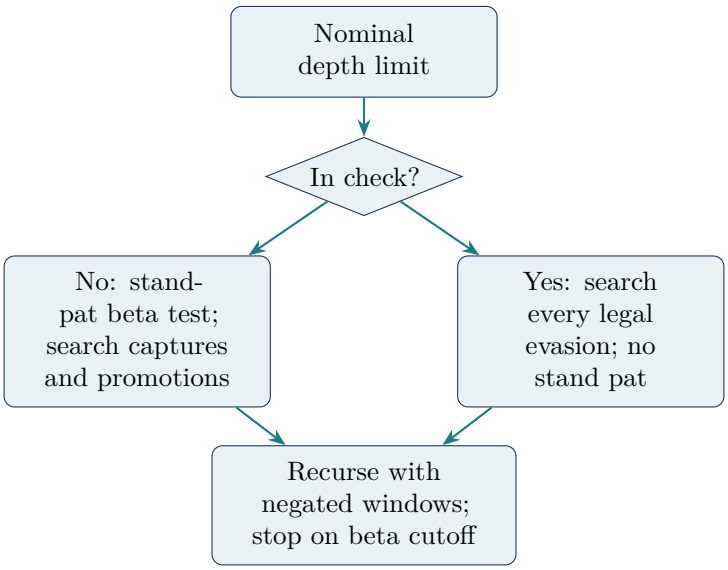


Figure 4.4: Quiescence uses stand pat and tactical moves in quiet positions, but searches every legal evasion when in check.

Listing 4.3: Simplified search-to-quiescence transition (pseudocode).

```
search(p, depth, alpha, beta):
    if depth == 0: return quiescence(p, alpha, beta)
    for move in ordered_legal_moves(p):
        make(move); score = -search(child, depth-1, -beta, -alpha); unmake(move)
        alpha = max(alpha, score)
        if alpha >= beta: break
    return alpha

quiescence(p, alpha, beta):
    if in_check(p): candidates = legal_evasions(p)
    else:
        stand_pat = evaluate(p)
        if stand_pat >= beta: return beta
        alpha = max(alpha, stand_pat); candidates = captures_and_promotions(p)
    search candidates recursively; return best score
```

The sketch omits repetition, mate-distance scoring, and termination guards.

4.3 Move ordering

Move ordering follows the stages in Table 4.2. MVV–LVA scores captures; threshold SEE (`see_ge`) alternates least-value attackers and updates x-rays as occupancy changes. At threshold zero, non-losing captures are searched first, losing captures are delayed in the main search, and losing captures are omitted from quiescence unless the side is in check. SEE is an ordering heuristic, not a legality test [6, 9].

Table 4.2: Move-ordering stages and limitations.

Stage	Purpose	Limitation
TT move	Try previously successful continuation first.	Validate legality; collisions/stale entries occur.
Promotions/captures	Queen and knight promotions rank first; SEE classifies captures with MVV–LVA tie-breaking.	Threshold SEE estimates local exchanges, not positional value.
Killer quiets	Reuse cutoff-producing quiet moves at same ply.	Position-specific usefulness varies.
History and countermove quiets	Order quiet moves using history, continuation-history, and countermove heuristics.	These scores are ordering hints, not position evaluations.

Scoring and sorting add cost; staged generation or partial selection may help. Measure total search performance—ordering alone does not guarantee strength.

4.4 Zobrist hashing and transposition table

Zobrist hashing XORs 64-bit keys for pieces, side to move, castling rights, and en-passant file; incremental updates are inexpensive. Each 16-byte transposition-table entry stores a key, depth, score, best move, bound type, and generation age; four entries share a 64-byte bucket. At sufficient depth, an exact score can answer a probe directly, while lower and upper bounds cut off only when they close the current window. Mate scores are normalized by ply when stored and probed. Otherwise, the table's move guides ordering; verification keys reduce but do not eliminate collisions [11, 10].

4.5 Time control and search termination

Search accepts depth, move-time, and side-to-move clock/increment limits. It derives soft and hard budgets, polls time, and retains a legal fallback and the last completed iteration. `stop` sets an atomic flag; the worker is joined before position changes or shutdown.

5. Position Evaluation

5.1 Handcrafted Evaluation

The evaluator combines material, Piece-Square Tables (PSTs), pawn structure, piece mobility, outposts, the bishop pair, and king safety. The evaluation is calculated from the perspective of the side to move, where positive scores indicate an advantage. To optimize performance, static features like material and PST totals are cached incrementally in the position state, while dynamic positional features are recalculated from the current board bitboards at each evaluation node [3, 7].

5.1.1 Material and Piece-Square Tables (PST)

The engine uses distinct middlegame (MG) and endgame (EG) weights for both raw material and piece positioning. Checkspeare utilizes the PeSTO evaluation tables to encourage rapid development, centralization, and sound opening principles.

Table 5.1: Implemented material weights.

Piece	MG	EG
Pawn	82	94
Knight	337	281
Bishop	365	297
Rook	477	512
Queen	1025	936
King	0	0

Each color's cached MG and EG totals combine these material weights with the PST values. FEN initialization calculates these from scratch, but during the search, moves update the score incrementally using the source and destination squares rather than rescanning the entire board. Captures subtract the victim's value, en passant uses the pawn's physical capture square, promotions replace the pawn's contribution with the new piece, and castling updates the rook's PST shift.

Undo records save these cached totals and the current game phase. The `unmake_move` function restores them directly from the history stack, preventing floating-point drift or accumulation errors during millions of search cycles.

Listing 5.1: Incremental material and PST update (simplified pseudocode).

```
move_piece(piece, color, from, to):
    mg[color] += PST_MG[piece][to] - PST_MG[piece][from]
    eg[color] += PST_EG[piece][to] - PST_EG[piece][from]

capture(piece, color, square):
    mg[color] -= MG_VALUE[piece] + PST_MG[piece][square]
    eg[color] -= EG_VALUE[piece] + PST_EG[piece][square]
    phase -= PHASE_WEIGHT[piece]
```

5.1.2 Pawn Structure

Pawn structure is evaluated independently for both colors using precomputed file and isolated masks.

- **Doubled Pawns:** The engine counts pawns on the same file. For every additional pawn beyond the first, a penalty of 12 MG and 20 EG is applied.
- **Isolated Pawns:** If a pawn lacks friendly pawns on both adjacent files, it is isolated and receives a penalty of 15 MG and 20 EG.
- **Passed Pawns:** A pawn is passed if no enemy pawns exist in front of it on its own file or adjacent files. Passed pawns receive a scaling bonus based on how far they have advanced, escalating dramatically on the 6th and 7th ranks (e.g., up to +100 MG and +160 EG for a pawn on the 7th rank).

5.1.3 Piece Mobility and Outposts

The engine uses a pawn-safe mobility count rather than simply counting pseudo-legal moves. A square is only considered a valid mobility target if it is not occupied by a friendly piece and is not attacked by an enemy pawn. Each safe square attacked by a knight, bishop, rook, or queen grants a piece-specific mobility bonus. Additionally, the engine evaluates **Knight Outposts**. A knight is considered an outpost if it satisfies three strict conditions:

1. It resides on a central advanced rank (ranks 4, 5, or 6 for White).
2. It is defended by a friendly pawn.
3. It cannot be chased away by an enemy pawn (i.e., no enemy pawns exist on adjacent files that could advance to attack the knight).

Knights achieving outpost status are rewarded with a substantial bonus (+20 MG, +10 EG). A flat bonus (+20 MG, +50 EG) is also awarded if a side possesses the **Bishop Pair**.

5.1.4 King Safety

King safety is dynamically evaluated by analyzing the pawn shield and tracking enemy pieces attacking the "King Zone."

- **Pawn Shield:** The engine applies a bitmask to the three squares immediately in front of the king. Each friendly pawn residing in this shield grants a defensive bonus (+15 MG, +5 EG).

- **King Zone Attacks:** The King Zone is defined as the king's current square, all adjacent squares, and the three squares pushed one rank forward.
- **Toga Formula:** If an enemy piece attacks any square in the King Zone, the engine increments an `attackers_count` and adds the piece's specific attack weight to an `attack_value` pool. Once all attacks are tallied, the engine scales the total danger using the Toga array formula:

$$\text{Danger} = \frac{\text{attack_value} \times \text{ATTACK_WEIGHTS}[\text{attackers_count}]}{100}$$

This ensures that a single piece attacking the king yields a negligible penalty, but coordinated attacks by multiple pieces result in an exponential drop in the evaluation score.

5.1.5 Tapered Score

Because piece values and positional priorities shift dramatically as pieces are traded, Checkspeare uses Tapered Evaluation to ensure smooth transitions between the middlegame and the endgame without rigid cutoffs.

The *Game Phase* is calculated by tracking the remaining non-pawn material. Pawns and Kings have a phase weight of 0, Knights and Bishops are 1, Rooks are 2, and Queens are 4. The maximum possible phase is 24 (the starting position).

The middlegame weight (w_{MG}) is derived as a fraction of 256, and the endgame weight (w_{EG}) takes the remainder:

$$w_{MG} = \frac{\text{Phase} \times 256}{24}, \qquad w_{EG} = 256 - w_{MG}$$

$$S = \frac{w_{MG} S_{MG} + w_{EG} S_{EG}}{256}$$

The final blended score is negated if it is Black's turn to move, ensuring that a positive evaluation always reflects a favorable position for the side currently searching. Search boundaries, forced mates, and exact 0-score draws are handled externally in the Alpha-Beta framework to prevent horizon interference.

6. Polyglot Opening Book

6.1 Purpose and integration

The opening book supplies candidates for known positions. Query it after position setup and return selected moves through the standard UCI response.

6.2 Record layout

A Polyglot record is 16 bytes: an 8-byte position key, 2-byte encoded move, 2-byte weight, and 4-byte learning field. Fields are big-endian and entries are sorted by key [12]. Table 6.1 shows the byte offsets.

Table 6.1: Polyglot 16-byte record layout.

Offset	Length	Field	Meaning
0–7	8	Key	Polyglot-specific 64-bit position key
8–9	2	Move	Packed source/destination/promotion
10–11	2	Weight	Candidate selection weight
12–15	4	Learn	Auxiliary learning value

Decode bytes explicitly to avoid native padding and endianness assumptions. Reject files whose length is not divisible by 16.

6.3 Keys, move validation, and selection

Polyglot has its own Zobrist constants; compute its key separately from the engine key. Binary-search matching records, choose by weight or maximum weight, then validate against legal moves, including castling, en passant, and promotions. Zero-weight or invalid candidates return no move, so search continues.

6.4 In-memory access and platform boundary

The book maps the file read-only: Windows uses a file handle, mapping object, and view; POSIX uses a descriptor and `mmap` [17, 21]. There is no buffered fallback in the current implementation.

Listing 6.1: Platform-specific read-only book mapping.

```
#if defined(_WIN32)
    // CreateFileMapping and MapViewOfFile
#else
    // open, fstat, and mmap
#endif
```

Validate file size, mapping success, and 16-byte alignment; bounds-check lookups and release resources once. Mapping uses the OS cache but does not guarantee data is resident or free.

7. UCI Protocol and Engine Communication

UCI is a line-based protocol for initialization, readiness, game setup, search, options, cancellation, and exit. Replies include handshake, analysis, and best-move messages. The engine advertises the `Hash`, `BookFile`, and `BookMode` options. Optional fields vary, and UCI defines communication, not chess algorithms [20].

7.1 Concurrent command handling and search

The main UCI loop reads commands while a worker searches. `stop`, `position`, `setoption`, and `quit` stop and join the worker before state changes; cancellation uses an atomic flag. Before `bestmove`, search reports depth, selective depth, nodes, NPS, score, and principal variation.

Listing 7.1: Engine UCI exchange during depth-limited search.

```
GUI -> Engine: uci
Engine -> GUI: id name Checkspeare
Engine -> GUI: option name BookFile type string default book.bin
Engine -> GUI: option name BookMode type combo default Weighted var Weighted var Best
Engine -> GUI: uciok
GUI -> Engine: isready
Engine -> GUI: readyok
GUI -> Engine: ucinewgame
GUI -> Engine: position startpos moves e2e4 e7e5
GUI -> Engine: go depth 12
Engine -> GUI: info depth 10 score cp 24 nodes 184203 pv g1f3 b8c6
Engine -> GUI: bestmove g1f3
GUI -> Engine: quit
```

Send newline-terminated commands and parse output line by line. `info` is analysis; `bestmove` ends a search; EOF signals process exit. Keep the GUI responsive, handle malformed messages, and associate results with the current game. Buffering, cancellation, and command order affect correctness.

Figure 7.1 shows asynchronous message delivery while the GUI thread remains responsive.

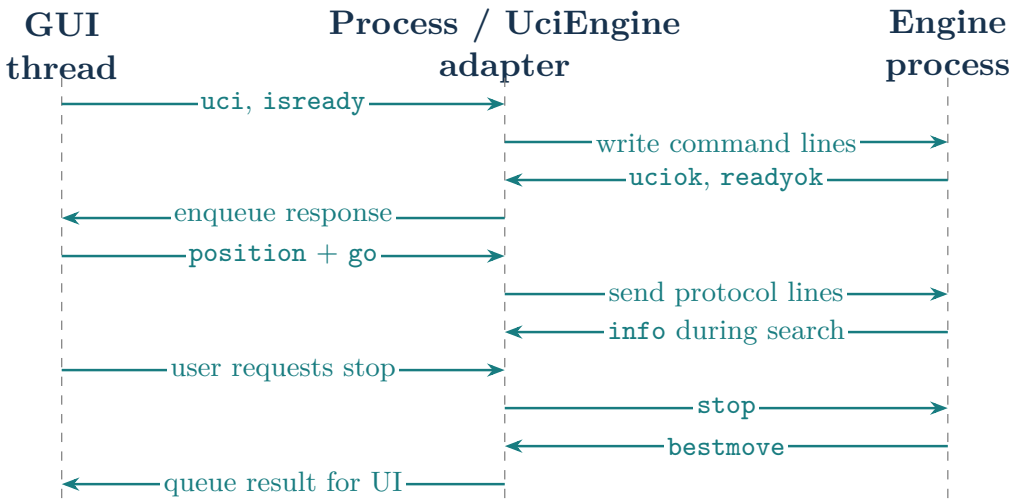


Figure 7.1: Asynchronous UCI exchange; the adapter delivers complete messages to the GUI thread.

Never apply `info` moves to the board; update analysis as it arrives, and commit moves only after user input or `bestmove`. The engine accepts `startpos` or `fen` with move lists, applies recognized legal moves, ignores unknown commands, and polls for cancellation at safe intervals.

8. GUI Architecture: SFML, Dear ImGui, and ImGui-SFML

8.1 Software Architecture

The GUI is organized around application orchestration, presentation, input, game state, and the engine-native chess core. Dependencies do not form a strict stack: the controller uses UI board-annotation types, and boundary APIs use SFML vectors. Figure 8.1 and Table 8.1 summarize the modules and dependencies.

8.1.1 Module Layout

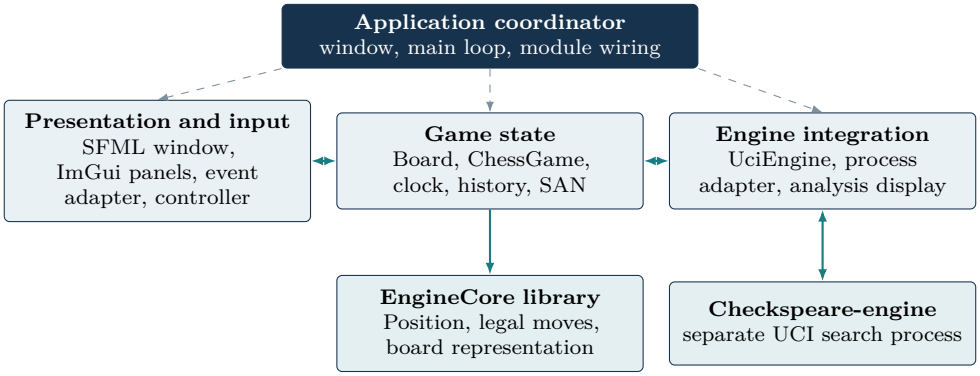


Figure 8.1: GUI modules and data paths. Solid arrows show game, search, rules, and UCI exchanges; dashed arrows show application coordination.

Module	Responsibility	Depends on
`engine/src/board/`	Engine-native chess rules: positions, move generation, and make/unmake; no GUI dependencies.	—
`game/`	`Board` wraps a `Position` and caches legal moves; `ChessGame` tracks history, cursor, and per-ply clock snapshots; `ChessClock` manages time control.	EngineCore
`engine/`	`UciEngine` handles UCI through the process adapter; each `EnginePlayer` state machine coordinates one side's search and applies returned moves.	`game/`, EngineCore, OS process adapter
`input/`	`SfmlInputAdapter` converts SFML events to semantic pointer/focus events; coordinates use `sf::Vector2f`.	SFML Window/System
`controller/`	Translates input to game actions; owns drag state, promotion selection, annotations, and queued commands.	`game/`, `input/`, board geometry and annotations
`ui/`	ImGui/SFML panels display the board, clocks, move list, captured pieces, status, evaluation, setup, and engine analysis.	`game/`, `controller/`, assets, ImGui/SFML
`audio/`, `assets/`	`assets/` loads textures and sound variants; `SoundPlayer` plays categorized cues.	SFML Graphics/Audio
`Application`	Owns the window and top-level state, runs the loop, wires callbacks, and coordinates modules and both engine/player pairs.	All application modules

Table 8.1: Module responsibilities and dependencies; some cross the conceptual layers in Figure 8.1.

8.1.2 Design Decisions

Callback-based clock synchronization. `ChessGame` exposes `ClockSync` for cursor changes and `MoveCommitted` for new moves. `Application` wires both: navigation restores the matching clock snapshot; a committed move updates the clock, saves a snapshot, and triggers move audio. It also supplies `humanMayMove` and `sideIsEngineControlled` predicates to the controller, which gates move commits and resignation without owning engine state.

Snapshot-based time travel. Each `MoveRecord` stores its post-move clock snapshot; `ChessGame` stores the initial snapshot. Undo, redo, and `gotoPly` use make/unmake and restore the snapshot at the new cursor. Positions are not duplicated as per-ply FEN strings.

Pure engine-player decision core. The pure `decideEngineAction(EnginePlayerInputs)` function chooses start, abort, finish, or idle. It is unit-testable without process I/O; `EnginePlayer` applies its action and communicates with `UciEngine`.

Scoped SFML input translation. `SfmlInputAdapter.hpp` converts supported SFML pointer and focus events to GUI events. SFML also appears in windowing, rendering, assets, audio, and coordinate types, so replacing it would affect more than the adapter.

Application as coordinator. `Application` coordinates both `UciEngine`/`EnginePlayer` pairs, setup, clock, game, controller, and views. It maps committed moves and game results to sound cues and advances `MoveAnimation`; the setup modal, `SoundPlayer`, and board panels remain separate components.

8.2 Responsibilities and layout

SFML manages the window, events, and rendering; Dear ImGui provides controls and panels through ImGui-SFML [19, 18]. The board displays local `ChessGame` state backed by EngineCore; the UCI client handles search-process messages. Figure 8.2 shows the interface.

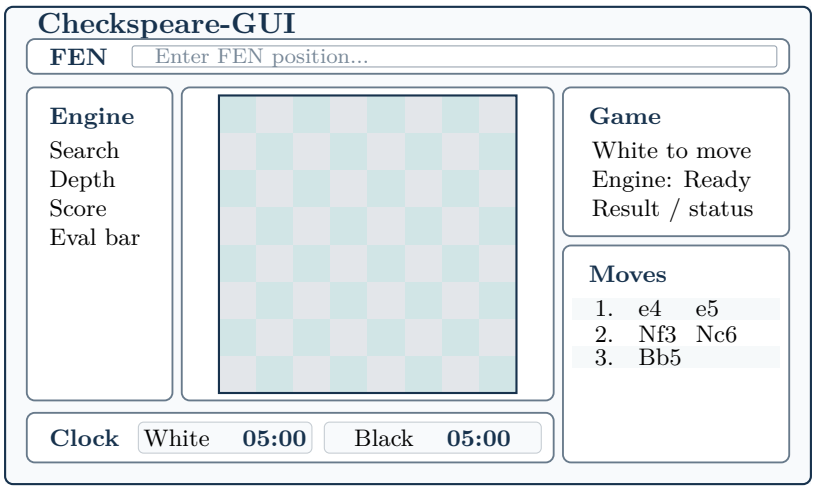


Figure 8.2: Schematic GUI layout with FEN input, engine controls, board, game status, move list, and clocks.

8.3 Board rendering and user interaction

For board origin (x_0, y_0) and square size s, map point (x, y) to file $\lfloor (x - x_0)/s \rfloor$ and rank $7 - \lfloor (y - y_0)/s \rfloor$ when White is at the bottom. Flip display coordinates for Black without changing engine state. Render pieces and highlights from application state.

Two clicks select a source and destination. The GUI validates and applies the move through EngineCore, then sends the position and move history to the search process. Keep history and game status in application state, and honor ImGui capture flags to prevent panel clicks from reaching the board.

8.4 Frame loop and responsiveness

Each frame processes events, restarts the delta clock, calls `update`, then calls `render`. Event processing forwards events to ImGui-SFML, handles window closure, translates supported events, and applies controller, modal, game-over, and ImGui capture checks. Pointer coordinates are converted to board space before dispatch; focus loss and release of an active drag have explicit paths.

`update` runs simulation before UI. Simulation consumes controller requests, applies queued actions, ticks engines when setup is closed, updates annotations and move animation, handles game-over transitions, and advances the clock when the game is active. The UI step builds the engine-thinking snapshot, updates the evaluation bar and ImGui, then renders the board and panels. `render` clears the window, renders ImGui, and displays the frame.

Listing 8.1: Application frame-loop pseudocode.

```
while window.isOpen():
    processEvents()
    dt = deltaClock.restart().asSeconds()
    update(dt)
    render()

update(dt):
    stepSimulation(dt)
    stepUi(dt)

stepUi(dt):
    snapshot = engines.buildThinkingSnapshot(game)
    updateEvalBar(snapshot, dt)
    ImGuiSFML.Update(window, seconds(dt))
    beginMainDockspace()
    renderBoardAndPanels(snapshot)

render():
    window.clear(White)
    ImGuiSFML.Render(window)
    window.display()
```

9. Cross-Platform Engine Subprocess Management

9.1 Shared process interface

The GUI separates process control (`Process`) from UCI parsing (`UciEngine`). The process API provides start, byte I/O, timed wait, termination, reset, and state queries. POSIX and Win32 adapters hide native handles (Figure 9.1).

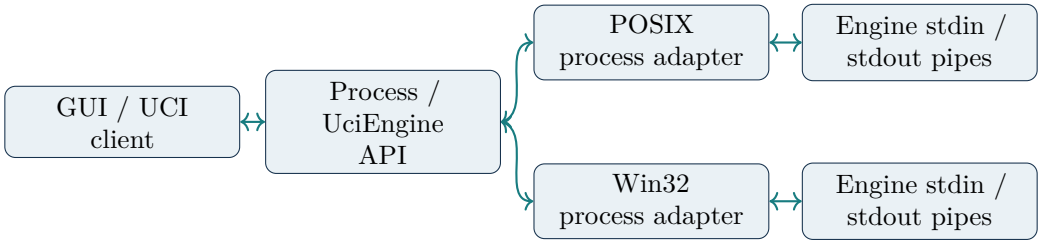


Figure 9.1: Shared process API with target-specific implementations.

9.2 POSIX process creation

The POSIX adapter creates input/output pipes and launches with `posix_spawn`. File actions connect child stdin, merge stdout/stderr, and close redundant child descriptors; the parent retains the input writer and output reader [22, 23].

Reads and writes may be partial: retry interrupted calls, complete writes, and assemble reader bytes into newline-delimited messages. Shutdown sends `quit`, polls `waitpid` for two seconds, then terminates and reaps the child if needed.

9.3 Win32 process creation

On Windows, create anonymous pipes, mark only child handles inheritable, set standard handles in `STARTUPINFO`, and launch with `CreateProcess` [16, 15]. Close process, thread, and pipe handles on every path. Both adapters implement `Process` and merge child stderr into stdout. Compiler support remains a separate build concern.

9.4 Background reads and thread safety

After launch, one `UciEngine` reader thread assembles stdout into protocol lines and queues them for nonblocking GUI polling.

On EOF, the reader records process exit. Cancellation sends `stop`; shutdown sends `quit`, waits two seconds, terminates on timeout, joins the reader, and releases resources. Listing 9.1 summarizes the lifecycle.

Listing 9.1: Reader-thread ownership and shutdown (pseudocode).

```
process.start(executable);
reader = std::thread([this] { readOutputUntilEof(); });

writeLine("quit");
if (!process.waitForExit(2s)) process.terminate();
if (reader.joinable()) reader.join();
process.reset();
```

The process layer handles partial writes and interrupted reads; UCI parsing occurs above it. Standard error shares stdout's pipe, and `stop`/`quit` share the input writer.

10. C++ Portability, Conditional Compilation, and CMake

10.1 Build targets and platform adapters

The project requires C++23. CMake builds the common static `EngineCore` library, the `CheckspeareEngine` engine executable, and the `CheckspeareGUI` application. SFML, Dear ImGui, ImGui-SFML, and Catch2 are fetched as build dependencies. Windows uses Win32 process and file-mapping APIs; Unix-like targets use POSIX implementations.

Table 10.1 lists the validated compiler and platform configurations.

Table 10.1: Validated compiler/platform matrix.

Compiler	Windows	Linux / Unix-like
GCC	Pass (MinGW)	Pass
Clang	Pass	Pass
MSVC	Pass	Not native

The compiler and operating system are separate dimensions. Process and book access use OS-specific source paths; the core chess code is shared between the engine and GUI.

10.2 Compile-time sliding-attack backend and CPU baseline

CMake enables `USE_PEXT` by default. PEXT uses BMI2 tables on single-architecture x86-64 targets; on other targets CMake falls back to HQ. Set `USE_PEXT=OFF` to select HQ explicitly. This choice is made at compile time, with no runtime CPU-feature detection.

For GCC, Clang, and AppleClang PEXT builds, `ENGINE_X86_ARCH` sets the `-march` target and defaults to `x86-64-v3`. This default applies only to these PEXT builds, not HQ builds [13]. Setting it to `native` targets the build machine; leaving it empty uses `-mbmi2` without an `-march` setting. MSVC-front-end PEXT builds use `/arch:AVX2`; clang-cl also receives `-mbmi2`. `ROOK_COMPRESSED` has effect only when PEXT is enabled.

10.3 CMake configuration

CMake keeps the engine and GUI executables distinct while linking both to the common static `EngineCore` library. Setting `BUILD_SHARED_LIBS=OFF` also requests static builds of fetched dependencies where those projects honor the setting. The `configure_portable_runtime` module applies additional runtime-link options only to non-imported executable targets, and only for configurations selected by `IS_PRODUCTION_CONFIG`: Release, RelWithDebInfo, MinSizeRel, or any configuration when `PRODUCTION_BUILD=ON`. Compiler warning flags remain target-scoped.

Table 10.2: Production runtime-link policy.

Platform	Policy
Windows (MSVC / clang-cl)	Static CRT in production (`/MT`, `/MTd` for Debug); DLL CRT otherwise (`/MD`, `/MDd`).
Windows (MinGW)	Production links detected GCC/C++ runtimes and dependencies statically; Windows system DLLs remain dynamic.
Linux / Unix	Static GCC/C++ runtimes. The engine tries full `-static` when supported; the GUI keeps glibc/OpenGL dynamic for driver loading.
macOS	Static system runtime unavailable; minimum deployment target is 11.0.

Static linkage is best-effort, not a guarantee of a self-contained executable: system libraries and non-Windows `libc++` may remain dynamic unless the toolchain is configured explicitly.

11. Testing and Validation

Table 11.1: Engine and GUI test coverage and reported results.

Area	Coverage	Result
Engine: perft	Known node counts from the initial position and standard test positions at multiple depths	Pass
Engine: legal moves	Checks, pins, king safety, castling, en passant, promotions, checkmate, and stalemate	Pass
Engine: position and hashing	Position initialization, move make/unmake including special moves, state restoration, and Zobrist consistency	Pass
Engine: move encoding	Source/destination/flag round trips, sentinel values, and promotion decoding	Pass
GUI: clock	Configuration, ticking, increments, pause/reset behavior, and flag fall	Pass
GUI: game state and notation	Legal play, move history navigation, game results, and SAN formatting	Pass
GUI: engine lifecycle	Engine action decisions and game-over transitions	Pass

12. Performance Evaluation and Experimental Results

12.1 Test environment

Measurements were collected on an AMD Ryzen 5 5600X system with 16 GB of RAM running Windows 11. The engine used a MinGW-w64 UCRT GCC 14.2.0 Release build with `-O3` and IPO/LTO. Search and match runs used a 64 MB hash. The PEXT and Hyperbola Quintessence (HQ) backends were measured separately.

12.2 Match against Stockfish 19 at 2900 Elo

Checkspeare played Stockfish 19 limited to UCI Elo 2900 (`SF2900`) in a 600-game CuteChess match. Each game used a 60-second base time with a 0.6-second increment and the Komodo opening book through depth 12. Stockfish used one thread. The match ran four games concurrently and enabled recovery, resignation, and draw adjudication. Its PGN, `sf2900.pgn`, is included in the project's `docs` directory.

Listing 12.1: CuteChess CLI configuration for the 600-game SF2900 match.

```
cutechess-cli.exe ^
  -engine name=Checkspeare cmd=CheckspeareEngine.exe option.Hash=64 ^
  -engine name=SF2900 cmd=stockfish.exe option.Hash=64 option.Threads=1 option.UCI_LimitStrength=true
    option.UCI_Elo=2900 ^
  -each proto=uci tc=60+0.6 book=komodo.bin bookdepth=12 ^
  -games 2 -rounds 300 -repeat ^
  -concurrency 4 -recover ^
  -resign movecount=3 score=800 twosided=true ^
  -draw movenumber=40 movecount=8 score=10 ^
  -ratinginterval 10 ^
  -pgnout sf2900.pgn
```

The 300 rounds with two games per round produce 600 games. CuteChess marked 464 games (77.3%) as adjudicated. Checkspeare recorded 125 wins, 210 draws, and 265 losses (230 of 600 points; 38.33%). BayesElo estimated SF2900 at $+39 \pm 12$ and Checkspeare at -39 ± 12, a 78-Elo gap.

Converting the observed score separately gives

$$\Delta R = 400 \log_{10}\left(\frac{0.3833}{1 - 0.3833}\right) \approx -83.$$

If SF2900 is calibrated at 2900 Elo under comparable conditions, this corresponds to a conditional performance estimate of about 2817 Elo. A rough 95% sampling interval is 2793–2841 Elo. It reflects match-result sampling only; it does not include uncertainty in SF2900's calibration, engine settings, hardware, or adjudication rules.

12.2.1 Color and opening breakdown

The color split in Table 12.1 shows a clear difference between Checkspeare's results as White and Black.

Table 12.1: Match results by Checkspeare's color.

Color	Games	W–D–L	Score
White	300	73–126–101	45.33%
Black	300	52–84–164	31.33%

Table 12.2 breaks down the results by opening family. The notably low score in the Sicilian Najdorf subset, particularly as Black, is the clearest weakness in this test suite; the limited opening distribution does not establish a general weakness against all Sicilian variations.

Table 12.2: Results by opening family.

Opening family	Games	Score
Queen's Pawn Game (A46)	300	45.67%
Sicilian Najdorf (B90)	234	27.14%
Caro–Kann family	66	44.70%

In the 234 B90 games, Checkspeare scored 36.32% as White and 17.95% as Black:

Table 12.3: Checkspeare's score by color in the Sicilian Najdorf subset.

Checkspeare's color	Games	Score
White	117	36.32%
Black	117	17.95%

12.3 Depth-six perft

Table 12.4: Initial-position depth-six perft results.

Measure	PEXT	HQ
Nodes	119,060,324	119,060,324
Elapsed time	752 ms	1,029 ms
Throughput (M nodes/s)	158.3	115.7
Relative throughput (PEXT/HQ)	1.37×	

12.4 Depth-18 search

Table 12.5: Initial-position depth-18 search throughput.

Measure	PEXT	HQ
Search depth	18	18
Throughput (M nodes/s)	5.11	2.67
Relative throughput (PEXT/HQ)	1.91×	

12.5 Summary

PEXT delivered higher measured throughput than HQ on both workloads: 1.37× for depth-six perft and 1.91× for depth-18 search. Measurements are specific to the stated hardware and build; the match result applies to the stated Stockfish settings, opening book, and time control.

13. Limitations, Future Improvements, and Conclusion

Checkspeare-engine combines bitboard move generation, compile-time PEXT/HQ sliding attacks, iterative alpha-beta search, SEE-based capture classification, a handcrafted tapered evaluator with incremental material/PST caches, and optional Polyglot book moves. Checkspeare-GUI uses EngineCore for local game state and communicates with the search executable through UCI pipes. The shared library and process protocol divide local interaction from engine search.

Search is single-threaded: the UCI loop runs it on a worker thread to remain responsive, but the search does not parallelize across cores. Evaluation is handcrafted rather than NNUE or another neural-network model. The GUI supports game setup and engine selection, but does not expose the engine's UCI options or support PGN import and export. Future work could add parallel search, neural evaluation, configurable UCI options, and PGN interoperability. Portability could improve with runtime CPU-feature dispatch that selects PEXT on BMI2-capable x86-64 CPUs and HQ otherwise; reproducible benchmarks across supported builds would strengthen performance comparisons.

Bibliography

[1] Chess Programming Wiki. Alpha-beta.

[2] Chess Programming Wiki. Bitboards.

[3] Chess Programming Wiki. Evaluation.

[4] Chess Programming Wiki. Hyperbola quintessence.

[5] Chess Programming Wiki. Move generation.

[6] Chess Programming Wiki. Move ordering.

[7] Chess Programming Wiki. Piece-square tables.

[8] Chess Programming Wiki. Quiescence search.

[9] Chess Programming Wiki. Static exchange evaluation.

[10] Chess Programming Wiki. Transposition table.

[11] Chess Programming Wiki. Zobrist hashing.

[12] Michel Van den Bergh. Polyglot book format. Format notes.

[13] Free Software Foundation. x86 options. GCC Online Documentation.

[14] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual*. Intel Corporation. BMI2 instruction reference.

[15] Microsoft. Createprocessw function. Windows Win32 documentation.

[16] Microsoft. Creating a child process with redirected input and output. Windows Win32 documentation.

[17] Microsoft. File mapping. Windows documentation.

[18] SFML Project. Imgui-sfml: Dear imgui backend for sfml.

[19] SFML Project. Sfml documentation.

[20] Stefan Meyer-Kahlen. Universal chess interface (uci), 2000. Protocol specification.

[21] The Open Group. mmap — map pages of memory. The Open Group Base Specifications, Issue 8.

[22] The Open Group. pipe — create an interprocess channel. The Open Group Base Specifications, Issue 8.

[23] The Open Group. posix_spawn — spawn a process. The Open Group Base Specifications, Issue 8.